

Boosted: Edzés menedzsment webalkalmazás

Szegedi Tudományegyetem Informatikai Intézet

SZAKDOLGOZAT

Földes Balázs

2026

Boosted: Edzés menedzsment webalkalmazás

Szegedi Tudományegyetem Informatikai Intézet

Boosted:

AI-támogatott fitness-ökoszisztéma és proaktív edzői platform

Szakdolgozat

Készítette:

Földes Balázs

programtervező
informatikus szakos
hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi adjunktus

Szeged

2026

Feladatkiírás

A fitness- és személyi edzői piacon számos edzésnaplózó alkalmazás érhető el, azonban kevés olyan komplex megoldás létezik, amely valódi, proaktív hidat képez az edző és a kliens között. A feladat egy olyan webalkalmazás készítése **React** frontend és **FastAPI/Python** backend technológiák használatával, ahol az edzők és klienseik strukturált keretek között tudják menedzselni a közös munkát.

A kliensek számára a rendszer lehetőséget biztosít a napi edzések, valamint az életmódbeli adatok (alvásidő, hidratáció, stressz-szint) rögzítésére, ezzel elősegítve a tudatos fejlődést. Az elkötelezettséget egy beépített széria-rendszer (streak) és egy interaktív motivációs eszköz (Boost) támogatja.

Az edzői oldal központi eleme a portfólió-menedzsment, ahol az edző valós időben követheti kliensei állapotát és statisztikáit. A szoftver kiemelt funkciója az **adatvezérelt riasztási rendszer**, amely algoritmusok segítségével automatikusan azonosítja a lemorzsolódási kockázatot vagy a kritikus túlterhelést.

Az alkalmazás innovatív megoldásként integrálja a **Gemini mesterséges intelligenciát**, amely a naplózott adatok alapján egyénre szabott, strukturált elemzéseket és javaslatokat generál mindkét felhasználói csoport számára. A jegyzetelési lehetőség és a prediktív AI-dashboard biztosítja, hogy az edzők gyorsabban és hatékonyabban tudjanak reagálni a klienseik fizikai és mentális állapotára, minimalizálva a sérülésveszélyt és maximalizálva az eredményességet.

Tartalmi összefoglaló

Téma megnevezése:

Boosted: AI-támogatott fitness-ökoszisztéma és proaktív edzői platform

A megadott feladat megfogalmazása:

A feladat egy olyan komplex webalkalmazás létrehozása, amely proaktív hidat képez az edzők és klienseik között. A rendszer célja a strukturált portfólió-menedzsment biztosítása, a kliensek napi életmódbeli és biometrikus adatainak (alvás, stressz, hidratáció, edzés) rögzítése, valamint a mesterséges intelligencia (AI) bevonásával prediktív, személyre szabott elemzések és riasztások generálása az eredményesebb és biztonságosabb közös munka érdekében.

Megoldási mód:

A feladat megoldásához frontend oldalon a React keretrendszert (Next.js alapokon), backend oldalon pedig Python nyelvet és a FastAPI keretrendszert alkalmazom.

Alkalmazott eszközök, módszerek:

A fejlesztés modern integrált fejlesztőkörnyezetben (pl. Visual Studio Code) történik. Kliensoldalon a React (Next.js) és a Tailwind CSS biztosítja a gyors és modern felhasználói felületet, míg a grafikonok megjelenítéséhez a Recharts könyvtárat, a dokumentumok exportálásához pedig a jsPDF-et alkalmazom. A szerveroldali logikát és az API végpontokat a nagy teljesítményű FastAPI szolgáltatja, az adatbázis-kezelésről pedig SQLite gondoskodik.

Elért eredmények:

A webalkalmazást a ..???... hosting szolgáltatásával hostoltam, illetve GitHub-ra is feltöltöttem, ahol publikusan elérhető.

Az elkészült webalkalmazás egy magas minőségű, reszponzív platform, amely asztali gépen, tableten és mobilon egyaránt kompromisszumok nélkül használható. A beépített gamifikációs elemek (szériák, "Boost" jutalmazás) sikeresen növelik a kliensek elköteleződését, míg a dinamikus AI-elemzések és az automatizált pdf-alapú edzésterv-generálás jelentősen felgyorsítják és skálázhatóvá teszik a szakértői munkát.

Kulcsszavak:

Tartalomjegyzék

Bevezetés

Az egészségtudatos életmód és a személyi edzés piacán egyre nagyobb az igény az adatalapú, transzparens közös munkára. Jelenleg számos edzésnaplózó alkalmazás érhető el, de kevés olyan átfogó megoldás létezik, amely proaktív hidat képez az edző és a kliens között. Ezt az űrt hivatott betölteni a Boosted webalkalmazás, amely egyetlen közös platformon teszi lehetővé a biometrikus adatok naplózását és a szakértői portfólió strukturált menedzselését.

Fontos szempont volt a webalkalmazás gyors és intuitív használata, hogy mind a szakértők, mind a kliensek a tényleges céljaikra tudjanak fókuszálni. Ezt a letisztult, modern felhasználói felület, a könnyen áttekinthető műszerfalak (dashboards) és a vizuális csúszkákkal támogatott gyors napi naplózás biztosítja. A kliensek percek alatt rögzíthetik az alvásidejüket, stressz-szintjüket, vízfogyasztásukat és edzésadataikat.

A hagyományos edző-kliens kapcsolatban sokszor elvesznek a kritikus információk két személyes találkozó között. A lemorzsolódás vagy a túledzés elkerülése érdekében az alkalmazás egy adatalapú, intelligens riasztási rendszert kapott. Az edzők azonnali, kiemelt értesítést látnak a műszerfalukon, ha egy kliens stressz-szintje kritikussá válik, vagy ha napok óta nem rögzített adatot, így időben, proaktívan be tudnak avatkozni a programba.

A pusztán adatrögzítésen túl a platform innovációja a mesterséges intelligencia integrációjában rejlik. A Boosted Neural Engine folyamatosan elemezi a kliensek naplóját, rejtett mintázatokat és trendeket keresve. Ezek alapján személyre szabott, prediktív elemzéseket és azonnal beépíthető javaslatokat generál a kiégés megelőzésére és a fejlődés optimalizálására, tehermentesítve ezzel az edzőt és maximalizálva a kliens eredményességét.

A szakmai útmutatás átadására a webalkalmazás egy interaktív heti edzéstervező modult kínál. A szakértők napra és időszakra lebontva tudják elkészíteni a programokat, amelyeket a kliensek a saját felületükön letisztult formában olvashatnak, a beépített exportáló funkciónak köszönhetően pedig akár PDF formátumban le is tölthetik azokat offline használatra.

A tartós életmódváltás egyik legnagyobb kihívása a motiváció fenntartása. A fiatalabb generációk és a modern applikációk elvárásainak megfelelően a Boosted is beépített gamifikációs elemekkel ösztönzi a felhasználókat. A folyamatos adatkitöltést a

rendszer "széria" (streak) számlálóval díjazza, míg az edzők egy egyedi, vizuális "Boost" küldésével tudják azonnal, pozitívan megerősíteni a klienseik kitartását.

A webalkalmazás reszponzivitásán is nagy volt a hangsúly. Cél volt, hogy a kliensek az adataikat bárhol – akár útközben vagy az edzőteremben – kényelmesen rögzíthessék mobiltelefonról, míg az edzők az összetett statisztikákat asztali gépen vagy tableten is átláthatóan tudják elemezni.

1. Piackutatás

A szakdolgozatom elkészítésének legelején területi áttekintést, piackutatást végeztem inspirálódásként. Az áttekintés során olyan webalkalmazásokat kerestem és tekintettem meg, amelyeknek célja a személyi edzők és klienseik közötti kommunikáció támogatása, a fejlődés nyomon követése, valamint az edzéstervezés és az életmód-naplózás megkönnyítése.

		Trainerize	MyFitnessPal	TrainingPeaks	Boosted
FUNKCIÓK	Kliens portfólió menedzsment	TRUE	FALSE	TRUE	TRUE
	Heti edzéstervező naptár	TRUE	FALSE	TRUE	TRUE
	Életmód naplózás (alvás, stressz, víz)	FALSE	TRUE	FALSE	TRUE
	AI-alapú prediktív elemzés	FALSE	FALSE	FALSE	TRUE
	Gamifikáció (Szériák, Boostok)	FALSE	FALSE	FALSE	TRUE
	Edzői figyelmeztetések (Túlterhelés)	FALSE	FALSE	TRUE	TRUE
	PDF edzésterv exportálás	FALSE	FALSE	FALSE	TRUE
	Modern, letisztult UI (Glassmorphism)	FALSE	FALSE	FALSE	TRUE
	Zárt, meghívó-alapú csatlakozás	TRUE	FALSE	TRUE	TRUE
	Privát edzői jegyzetek és profilozás	TRUE	FALSE	TRUE	TRUE
	Vizuális adatelemzés (Grafikonok)	TRUE	TRUE	TRUE	TRUE
	AI Chat Asszisztens	FALSE	FALSE	FALSE	TRUE

1.1.ábra: Összefoglaló táblázat az áttekintett webalkalmazásokról.

Az 1.1. ábrán látható táblázat a területi áttekintés során készült összefoglaló táblázat, amely az áttekintett webalkalmazások funkcionalitását foglalja össze. A *TRUE* jelzi, ha a webalkalmazásban megtalálható az adott funkcionalitás, a *FALSE* jelzi, ha nem található meg az adott funkcionalitás.

1.1. Trainerize

A Trainerize[1] az egyik legszélesebb körben használt edzői platform, amely a személyi edzők és fitneszstúdiók számára készült. A szoftverben az edzők heti edzésterveket tudnak készíteni, videós gyakorlatskönyvtárat használhatnak, és nyomon követhetik a kliensek alapvető fizikai paramétereit (például testsúly, testzsírszázalék). A webalkalmazás és a hozzá tartozó mobilapplikáció használatához zárt, meghívó-alapú regisztráció szükséges. Hátránya, hogy a rendszer túlságosan a fizikai edzésmunkára fókuszál, így a holisztikus

életmód-naplózás (stressz-szint, alvásminőség) nem hangsúlyos benne. Továbbá nem rendelkezik beépített mesterséges intelligenciával, amely a begyűjtött adatokból proaktívan előrejelezné a kliens esetleges kiegészét, vagy segítené az edzőt a tervek optimalizálásában.

1.2. MyFitnessPal

A MyFitnessPal[2] egy globálisan népszerű, elsősorban végfelhasználóknak (klienseknek) készült alkalmazás. Fő profilja a táplálkozásnaplózás, a kalóriaszámlálás és az alapvető napi aktivitás (pl. lépésszám, vízfogyasztás) nyomon követése. A hatalmas, közösség által is bővített élelmiszer-adatbázisa miatt egyéni célok elérésére kiválóan alkalmas. A legnagyobb hiányossága azonban, hogy nem támogatja az edző-kliens modellt. Nem rendelkezik portfóliómenedzsment felülettel szakértők számára, így az edzők nem tudnak zárt rendszerben heti edzésterveket kiírni a klienseiknek. A rendszerből hiányzik a modern, kétirányú interakció és a gamifikáció is (például egy edzői "Boost" küldése), ami a motiváció fenntartásához elengedhetetlen lenne. A funkciók nagy része bejelentkezéshez, a részletesebb statisztikák pedig prémium előfizetéshez kötöttek.

1.3. TrainingPeaks

A TrainingPeaks[3] a professzionális sportolók, különösen az állóképességi sportágak (triathlon, kerékpár, futás) képviselőinek és edzőinek a legkedveltebb szoftvere. Rendkívül részletes adatokat, teljesítménygrafikonokat és túlterhelési figyelmeztetéseket biztosít, így az edzők mélyrehatóan tudják elemezni a sportolók állapotát. Lehetőség van privát edzői jegyzetek készítésére is. A rendszer hátránya annak extrém komplexitásából fakad. A felhasználói felülete (UI) rendkívül adatsűrű, sokak számára elavult hatást kelt, és a laikus, átlagos fitnesscélokkal rendelkező kliensek számára kifejezetten ijesztő és nehezen értelmezhető lehet. Ezenfelül hiányoznak belőle a modern LLM (Large Language Model) alapú AI asszisztensek, amelyek a nyers adatokat közérthető, szöveges javaslatokká formálnák.

3. Tervezett megjelenés

4. Felhasznált technológiák

A Boosted webalkalmazás fejlesztése során kiemelt szempont volt a modern, skálázható és nagy teljesítményű technológiák alkalmazása. A rendszer architektúrája élesen különválasztja a felhasználói felületet (frontend) és a szerveroldali logikát (backend), amelyeket REST API végpontok kötnek össze.

4.1. React és Next.js

A React egy nyílt forráskódú, komponensalapú JavaScript programkönyvtár, amelyet felhasználói felületek (UI) építésére terveztek. A projekt a Reactra épülő Next.js keretrendszert használja (amelyre a kódokban található use client direktívák is utalnak). Segítségével az alkalmazás moduláris, újra felhasználható komponensekből épül fel, ami biztosítja a gyors, oldalfrissítés nélküli navigációt (Single Page Application élmény) és a gördülékeny állapotér-kezelést (State Management).

4.2. Tailwind CSS

A Tailwind egy „utility-first” CSS keretrendszer, amely előre definiált, alacsony szintű stílusosztályokat biztosít a fejlesztők számára. A hagyományos CSS írása helyett ezeknek az osztályoknak a kombinálásával a HTML (JSX) kódon belül alakítható ki a dizájn. A Boosted reszponzív felülete, a modern Bento Grid elrendezés, valamint a prémium hatást keltő áttetsző felületek (Glassmorphism) és színátmenetek mind a Tailwind CSS segítségével készültek.

4.3. FastAPI (Python)

A szerveroldali (backend) logika megvalósításához a Python nyelven írt FastAPI keretrendszert alkalmaztam. A FastAPI az egyik leggyorsabb modern webes keretrendszer, amely aszinkron működést tesz lehetővé, és automatikus, interaktív API dokumentációt (Swagger UI) generál. Ez a réteg felel a felhasználói kérések (pl. bejelentkezés, adatok lekérése, mentése) villámgyors és biztonságos kiszolgálásáért.

4.4. SQLite és SQLAlchemy

Az alkalmazás adatbázis-kezelését a pehelysúlyú, relációs SQLite adatbázismotor látja el. Az adatbázis és a Python kód közötti kommunikációt az SQLAlchemy nevű ORM (Object-Relational Mapping) könyvtár biztosítja. Az ORM használatával az adatbázis-táblák (felhasználók, napi naplók, edzéstervek) Python objektumokként kezelhetők, ami biztonságosabbá és átláthatóbbá teszi az adatmódosítási műveleteket.

4.5. Google Gemini AI

A rendszer egyik legfőbb innovációja a Google mesterséges intelligencia szolgáltatásának, a Gemini AI-nak (konkrétan a *Gemini Flash* modellnek) az integrálása. A backend a Gemini API segítségével dolgozza fel a kliensek nyers biometrikus és életmód adatait. Az AI ebből prediktív kockázatelemzéseket (sérülés, kiégés), strukturált adathalmazokat (JSON) és szöveges, interaktív asszisztensi válaszokat generál az edzők és a kliensek számára.

4.6. Recharts

A Recharts egy React-alapú, kompozíciós elven működő diagramkészítő könyvtár. A webalkalmazásban az adatvizualizációért felel: ennek a segítségével jelennek meg az edzői és kliens műszerfalakon (dashboardokon) a letisztult, interaktív oszlop- és vonaldiagramok, amelyek az alvást, stresszt és edzésidőt ábrázolják az elmúlt napok tükrében.

4.7. jsPDF

A jsPDF egy nyílt forráskódú kliensoldali JavaScript könyvtár, amely lehetővé teszi PDF dokumentumok dinamikus generálását közvetlenül a böngészőben. A Boosted alkalmazásban az edzői modul használja: a rendszer ennek segítségével alakítja át a JSON formátumban tárolt heti edzésterveket formázott, olvasható, offline is használható PDF fájlakká.

4.8. Pydantic

A FastAPI backend elengedhetetlen része a Pydantic könyvtár használata. Ez a modul felel a kliens és a szerver közötti adatsere (request és response) szigorú típusellenőrzéséért és validációjáért. A Pydantic segítségével deklarált adatsémák (pl. regisztrációs űrlap, napi napló adatok) garantálják, hogy az adatbázisba és az AI modellhez csak tiszta, formátumilag helyes adatok kerülhessenek be.

4.9. Kriptográfiai modulok

A felhasználói adatok biztonsága kritikus szempont. A rendszer nem tárolja egyszerű szöveggént a jelszavakat; helyette a Python beépített hashlib könyvtárát használja SHA-256 alapú titkosításra (hashing). Emellett a zártkörű, meghívó-alapú regisztrációs rendszerhez a secrets könyvtár generál kriptográfiailag biztonságos, véletlenszerű URL tokeneket, megakadályozva az illetéktelen csatlakozásokat.

4.10. Uvicorn

A Python alapú FastAPI keretrendszer futtatásához egy aszinkron szerverátjáróra van szükség. Ezt a szerepet az Uvicorn tölti be, amely egy villámgyors ASGI (Asynchronous Server Gateway Interface) szerver. Ennek köszönhetően a backend képes egyszerre több kliens kérését (pl. AI generálás, adatbázis-műveletek) blokkolásmentesen, nagy teljesítménnyel kiszolgálni.

5. Rendszerarchitektúra

A webalkalmazás tervezése során egy modern, szétválasztott (decoupled) kliens-szerver architektúrát alkalmaztam. A rendszer logikailag és fizikailag is két fő komponensből áll: a felhasználói felületért felelős kliensoldalból (Frontend) és az üzleti logikát, valamint az adatkezelést végző szerveroldalból (Backend).

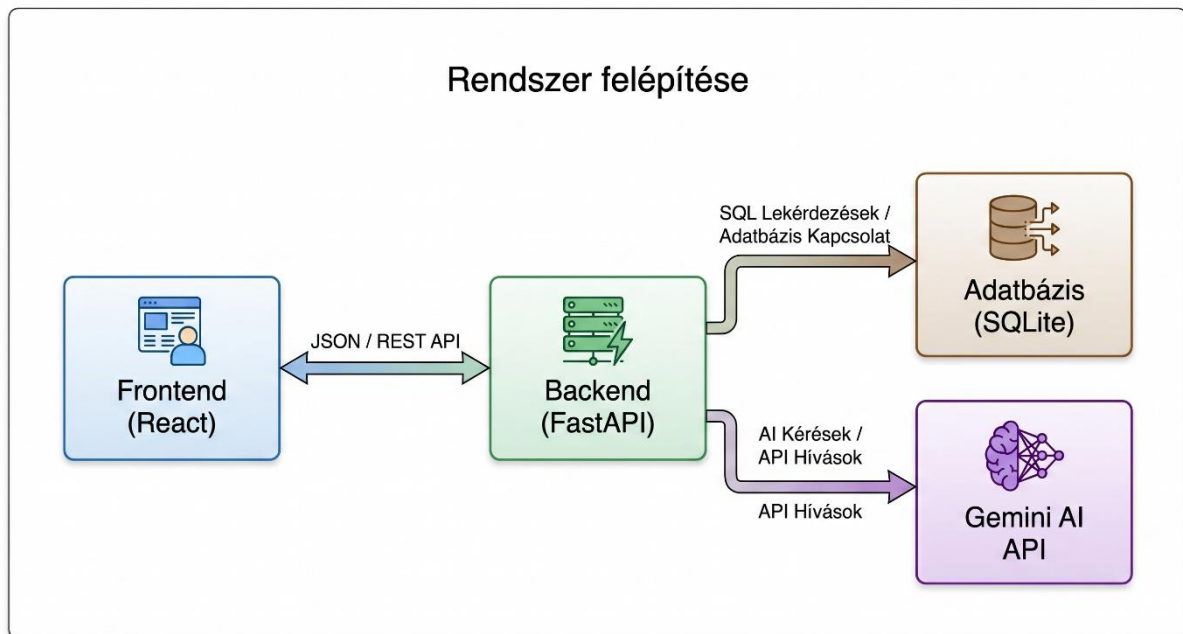
A két réteg közötti adatcsere a REST (Representational State Transfer) architekturális stílus elveit követve, aszinkron HTTP kéréseken keresztül valósul meg. Amikor a kliensoldalnak új információra van szüksége, vagy egy felhasználói interakció (például naplózás, profilmódosítás) történik, a Frontend megfelelő HTTP metódusok (GET, POST, PUT, DELETE) használatával JSON formátumú adatsomagokat küld a Backend felé. A szerveroldal feldolgozza a kérést, elvégzi a szükséges adatbázis-műveleteket vagy AI-generálást, majd a választ szintén strukturált JSON formátumban küldi vissza a prezentációs rétegnek.

5.1. Kliensoldal (Frontend)

A prezentációs réteg, amellyel a felhasználó – legyen az szakértő vagy kliens – közvetlen interakcióba lép. Ez a réteg felelős a vizuális megjelenítésért, az űrlapok kezeléséért, a kliensoldali validációkért és a dinamikus állapotkezelésért (State Management). A hagyományos, különálló HTML és SCSS fájlokkal operáló megközelítés helyett a rendszer a React JSX (JavaScript XML) szintaxisát használja, ahol a struktúra és a logika egyetlen moduláris komponensben egyesül. Az elemek stílusát és reszponzív viselkedését a Tailwind CSS keretrendszer utility-osztályai határozzák meg, amelyek közvetlenül a komponensek szintjén kerülnek definiálásra. Az alkalmazás működéséért és a dinamikus UI-váltásokért (például a modális ablakok megjelenítéséért vagy a diagramok újrarajzolásáért) a React Hook-jai (mint a useState és useEffect) felelnek.

5.2. Szerveroldal (Backend)

A rendszer "agya", amely az üzleti logikáért, a biztonságért és az adattárolásért felel. A FastAPI alapú Backend közvetlenül nem jelenít meg vizuális elemeket a felhasználó számára; kizárólag API (Application Programming Interface) végpontokat szolgáltat a Frontend számára. Ez a réteg felel a felhasználók hitelesítéséért (jelszavak hashelése, tokenek kezelése), az adatbázis-műveletekért (CRUD operációk), valamint az adatok séma-szintű validációjáért (Pydantic modellek). A Boosted alkalmazás szerveroldalának egyik legfőbb egyedi komponense a harmadik féltől származó AI-motorral (Google Gemini) való aszinkron kommunikáció. A Backend összegyűjti és anonimizálja az adatbázisból kinyert biometrikus adatokat, majd kontextusként átadja azokat a mesterséges intelligenciának, hogy a visszaérkező prediktív elemzéseket már feldolgozott formában juttassa el a kliensoldalra.



5.1.ábra: A webalkalmazás rendszerarchitektúrája és a komponensek közötti adatáramlás.

6. Belső felépítése

A webalkalmazás React (Next.js) alapokon nyugszik, amelynek legnagyobb előnye a komponens-alapú, deklaratív architektúra. Ennek köszönhetően a felhasználói felület logikailag jól elkülönülő, állapotvezérelt (state-driven) egységekből épül fel. Az alkalmazás nem használ hagyományos oldalújrátöltést; a navigációt és a különböző felületek megjelenítését a React állapotkezelő Hook-jai (pl. `useState`, `useEffect`) dinamikusan vezérlik.

A webalkalmazás logikai komponenseit két nagy csoportba lehet sorolni. Az egyik csoport a fő **Nézetek (Views)**, amelyek az egész képernyőt kitöltő, alapvető oldalakként funkcionálnak. A másik csoportba a **Modális és UI komponensek (Modals & UI Components)** tartoznak, amelyek a nézetek felett, felugró ablakokként jelennek meg, lehetővé téve a gyors interakciókat anélkül, hogy a felhasználó elveszítené az aktuális kontextust.

6.1. Fő nézetek (Views)

Ezek a komponensek határozzák meg a képernyő fő struktúráját attól függően, hogy a felhasználó éppen az alkalmazás melyik fázisában tart. A navigációt a view állapotváltozó szabályozza.

- **LandingView (Kezdőoldal):** A webalkalmazás nyilvános bemutatkozó felülete. Ez a komponens jelenik meg az alkalmazás megnyitásakor. Modern, úgynevezett "Bento Grid" elrendezéssel, reszponzív kártyákon keresztül mutatja be a szoftver előnyeit elkülönítve az edzők és a kliensek számára.
- **AuthView (Bejelentkezés és Regisztráció):** Ez a felület felel a felhasználók hitelesítéséért és a fióklétrehozásért. Különlegessége, hogy egyetlen dinamikus "Split-panel" elrendezést használ, amely vizuálisan zökkenőmentesen vált a belépési és a regisztrációs űrlapok között, interaktív visszajelzéseket adva a validáció során.
- **DashboardView (Vezérlőpult):** A bejelentkezett felhasználók központi felülete, amely a feltételes renderelésnek köszönhetően dinamikusan változik a felhasználó szerepköre (Edző vagy Kliens) alapján. Belső lapfüleket (Tabs) is tartalmaz:
 - *Áttekintés (Overview):* Edzők számára a klienslistát és a rendszerszintű statisztikákat, kliensek számára a napi teendőket és a naplókat jeleníti meg.

- *AI Asszisztens (AI)*: A Gemini API által generált prediktív elemzéseket és figyelmeztetéseket vizualizáló adatközpont.
- *Profil (Profile)*: A felhasználó saját adatainak (fizikai paraméterek, bemutatkozás) szerkesztésére szolgáló felület.
- **PremiumView (PRO Felület)**: Egy dedikált információs oldal, amely a Boosted PRO előfizetési csomag funkcióit (korlátlan AI asszisztens, PDF exportálás) mutatja be prémium vizuális elemekkel.
- **ClientDetailsView (Kliens Adatlap)**: Az edzői vezérlőpulton belül megnyíló, részletes nézet. Recharts grafikonokon ábrázolja a kiválasztott kliens fejlődését, továbbá tartalmazza az edzői privát jegyzetelés és a "Boost" küldés lehetőségét.

6.2. Modális és UI komponensek (Modals)

Ezek az újra felhasználható "Pure" komponensek felugró ablakokként (dialog) jelennek meg a fő nézetek felett, biztosítva a megszakításmentes felhasználói interakciót.

- **AppAlert**: Globális, dinamikus értesítő komponens, amely a backend műveletek eredményéről (siker, hiba) ad vizuális visszajelzést a szerepkörnek megfelelő színekkel.
- **DailyLogModal**: A kliensek napi adatbeviteli felülete vizuális csúszkákkal (alvás, stressz, víz, edzés).
- **WeeklyPlanEditorModal**: Az edzők számára készült heti edzéstervező felület, idősáv-megadással és egyéni idővonal-kezeléssel.
- **WeeklyPlanViewerModal**: A kliensek számára készült olvashatóság-fókuszú felület az edzéstervek megtekintésére.
- **LogDetailsModal**: A korábbi naplóbejegyzések részletes, kártyás megjelenítéséért felelős komponens.
- **InviteModal**: Kliensmeghívó tokenek generálását és e-mailes kiküldését kezelő ablak.
- **CoachProfileModal**: A kliens számára teszi elérhetővé a hozzá rendelt edző szakmai bemutatkozását és statisztikáit.

- **ProfilePictureModal:** Külső URL (pl. Imgur) alapján történő profilkép-módosító ablak.
- **PdfExportModal:** Lehetővé teszi az edző számára a korábbi tervek kiválasztását, majd a jsPDF könyvtár segítségével a tervek kliensoldali letöltését.
- **BoostNotificationModal:** A gamifikációs (Boost) jutalmak megjelenítésére szolgáló, egyedi animációval ellátott értesítő ablak.

6.3. API szolgáltatások (Backend Endpoints)

Az adatok kezeléséért és a kliens-szerver kommunikációért a FastAPI alapú backend RESTful végpontjai (endpoints) felelnek. Ezek a végpontok a frontend fetch hívásaira reagálnak, JSON formátumban.

- **Auth Service (/api/login, /api/register):** A bejelentkezést, a jelszavak hashelését (SHA-256) és a jogosultságok kezelését végzi.
- **Profile Service (/api/user/{id}/profile):** A felhasználói profiladatok (magasság, súly, célok, bemutatkozás) lekéréséért és frissítéséért felel.
- **Invite Service (/api/invite):** Kriptográfiailag biztonságos, egyedi tokeneket generál az új kliensek csatlakoztatásához.
- **Client Management (/api/coach/{id}/clients):** Visszaadja az edzőhöz tartozó kliensek listáját és azok legfrissebb heti tervét.
- **Daily Log Service (/api/client/log):** A napi biometrikus naplók adatbázisba írásáért és lekéréséért felelős végpontok.
- **Plan Service (/api/client/{id}/plan):** A heti edzéstervek (JSON stringként tárolt) mentését és módosítását végzi.
- **Boost Service (/api/client/{id}/boost):** A gamifikációs motivációs rendszer logikáját kezeli, növelve a kliens jutalompontjait.
- **AI Dashboard Service (/api/generate-ai-dashboard):** A Google Gemini API-val kommunikáló végpont, amely a rendszerszintű statisztikák alapján átfogó, JSON formátumú prediktív elemzést generál.

- **AI Client Analysis (/api/generate-client-analysis):** Kifejezetten az egyéni kliensek utolsó naplójából és az edzői jegyzetekből készít részletes kockázat- és célelemzést a Gemini modell segítségével.

6.4. Adatmodellek (Database Interfaces / Schemas)

A backendben az adatok szerkezetét és validációját az SQLAlchemy ORM modellek, valamint a Pydantic adatcsere-objektumok (Schemas) határozzák meg. Ezek garantálják az adatbázis (SQLite) integritását.

- **User Modell:** A felhasználók (edzők és kliensek) közös táblája. Tartalmazza a hitelesítési adatokat (email, password_hash), a fizikai paramétereket (height, weight), a szerepkört (role), valamint a gamifikációs oszlopokat (total_boosts).
- **DailyLog Modell:** A napi naplót (alvás, stressz, víz, edzésidő, hangulat, lépések) tároló, dátum alapú relációs tábla, amely idegen kulccsal (client_id) kapcsolódik a felhasználóhoz.
- **WeeklyPlan Modell:** A heti edzésterveket tároló tábla, amely JSON stringként perzisztálja az adott hét napjainak feladatait.
- **Invite Modell:** A kiküldött kliensmeghívókat (token, lejáratí idő, státusz) kezelő tábla.
- **Pydantic Sémák (pl. DailyLogCreate, ProfileUpdate):** A kliens felől érkező HTTP kérések (Request Body) adatainak típusellenőrzéséért és validálásáért felelős struktúrák.

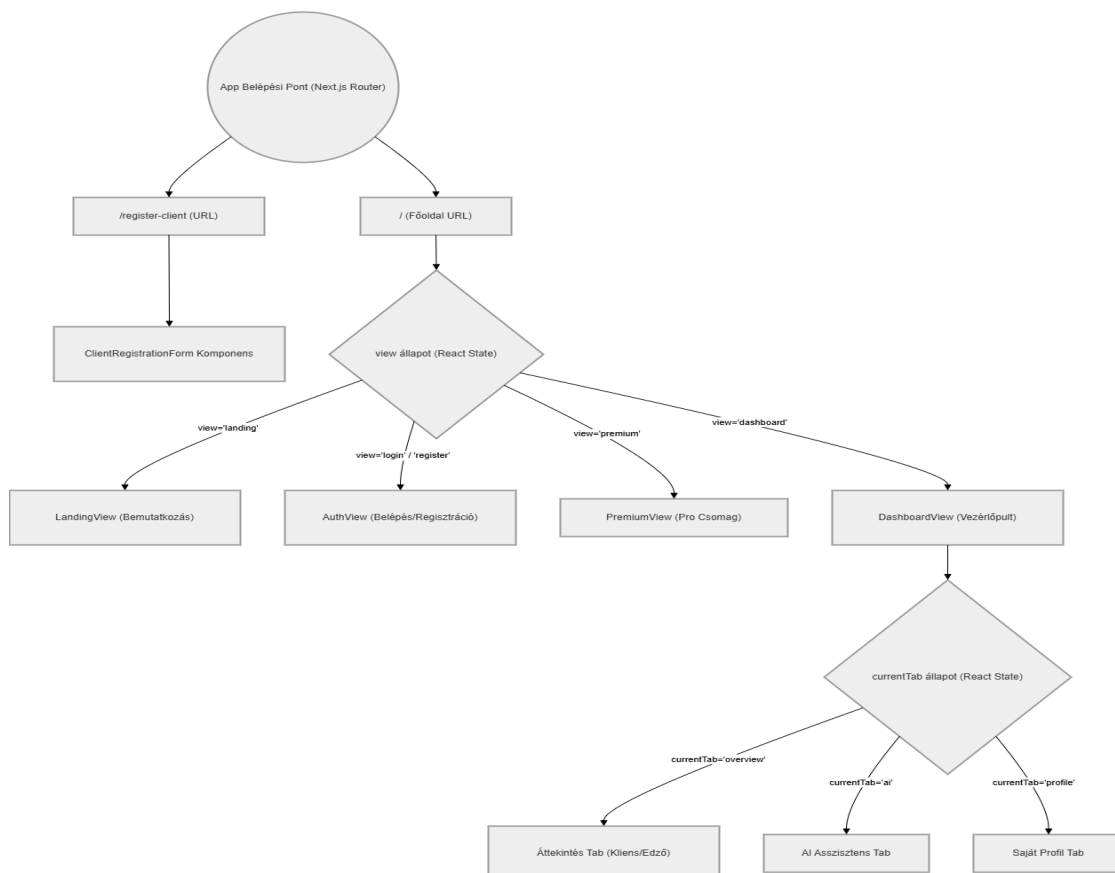
6.5. Adatbetöltési stratégiák (Data Fetching / Resolvers)

Az alkalmazás "Resolverek" helyett a React useEffect hookjait használja az adatok aszinkron betöltésére, még mielőtt a felhasználó megtekintené azokat:

- **Dashboard Data Loader:** A vezérlőpult betöltésekor automatikusan lekéri az edzőhöz tartozó klienseket, a napi statisztikákat, kliens esetén pedig a saját naplókat és edzésterveket.
- **AI Dashboard Loader:** Csak akkor fut le, ha a felhasználó az "AI Asszisztens" fülre navigál. Összegyűjti az aktuális kontextust, elküldi a Gemini AI-nak, majd a visszaérkező JSON adatokból felépíti az intelligens kártyákat.

6.6. Útvonalválasztás és Jogosultságkezelés (Routing & Guards)

A felületek közötti navigáció megoldására a webalkalmazás egy modern, hibrid megközelítést alkalmaz, amely egy belső állapotgépre (State Machine) és a Next.js kiegészítő eszközeire épül.



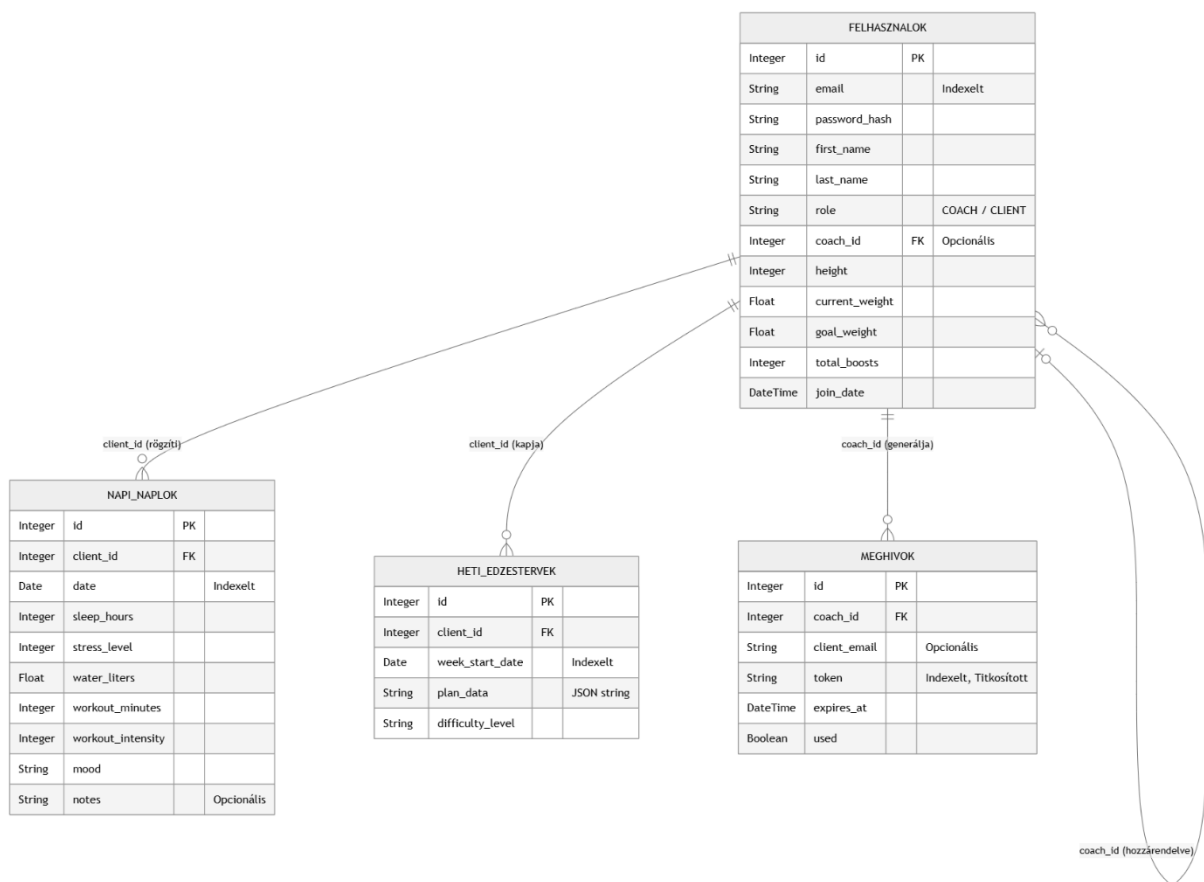
6.6.ábra: A webalkalmazás navigációs fája és nézethierarchiája.

A 6.6. ábrán is látható, hogy a rendszer logikailag több szintre tagolódik. A navigáció és a hozzáférés-vezérlés az alábbi három fő pilléren nyugszik:

- **Állapot-alapú Routing:** Az alkalmazás fő nézetei között a hagyományos oldalújrátöltés helyett a view állapotváltozó (landing, login, dashboard, premium) módosításával történik a váltás. A komponensek dinamikusan cserélődnek le az oldalon. A vezérlőpulton belül egy másodlagos állapot (currentTab) felel az aloldalak közötti lapozásért.
- **Auth Guard (Jogosultságkezelés):** A rendszer szigorúan ellenőrzi a felhasználó bejelentkezési státuszát és a userRole (COACH / CLIENT) állapotot. Az edzők nem férhetnek hozzá a kliens-naplózó felületekhez, a kliensek pedig nem láthatják az edzői statisztikákat és a portfóliókezelőt. Hitelesítés nélkül az alkalmazás a publikus nézeteken (Landing vagy Auth) tartja a felhasználót.
- **URL Paraméter Kezelés (useSearchParams):** Bár a fő alkalmazás Single Page Application-ként működik, a fizikai útvonalak is szerepet kapnak. A regisztrációs oldalon (Next.js Router használatával) az alkalmazás képes kiolvasni a meghívó linkekben szereplő titkosított tokeneket (?token=...), és ez alapján automatikusan azonosítani az új klienshez tartozó edzőt.

7. Adatmodell

A Boosted webalkalmazás adatainak perzisztens tárolásához relációs adatbázis-kezelő rendszert (SQLite) alkalmaztam. Az adatbázis és a szerveroldali üzleti logika (FastAPI) közötti kapcsolatot az SQLAlchemy ORM (Object-Relational Mapping) keretrendszer biztosítja. Szemben a NoSQL (például dokumentumalapú) megoldásokkal, a relációs adatmodellben az adatok szigorúan típusos, normalizált táblákban (táblázatokban) kerülnek eltárolásra. Az egyedek közötti összeköttetéseket idegen kulcsok (Foreign Keys) és azokon alapuló relációk definiálják, ami garantálja a rendszer magas szintű adatintegritását és a konzisztens lekérdezéseket.



7.ábra: A webalkalmazáshoz készült egyed kapcsolat diagramm.

7.1. Felhasználó (Users)

A users táblában a webalkalmazásba regisztrált összes felhasználó adata tárolódik. A rendszer egyetlen, közös táblát használ a szakértők (Edzők) és a Kliensek számára, a kettő közötti megkülönböztetést a role (szerepkör) mező végzi. A kliensek esetében a coach_id mező biztosítja a relációt a hozzájuk rendelt szakértő felé. Egy felhasználóról eltároljuk az

azonosításhoz szükséges email címet és a kriptográfiailag titkosított jelszót (SHA-256 password_hash), valamint a vezeték- és keresztnévet. Ezen felül a tábla számos opcionális profiltulajdonságot tartalmaz:

- **Fizikai és cél adatok:** Magasság, jelenlegi súly, célsúly, elsődleges cél, étrend és allergiák.
- **Szakértői identitás adatok:** Város, szakmai bemutatkozás (bio), tapasztalat évei, motivációs jelige, csatlakozás dátuma, és a profilkép URL-je.
- **Gamifikációs adatok:** Eltároljuk a kliens által összegyűjtött motivációs pontokat (total_boosts), valamint egy logikai (boolean) értéket, ami jelzi, ha a kliensnek olvasatlan jutalma van (has_unseen_boost). A felhasználók a saját adataikat szabadon módosíthatják, a jelszavak tárolása pedig egyirányú titkosítással történik.

7.2. Napi napló (DailyLogs)

A daily_logs táblában a kliensek által napi szinten rögzített biometrikus és életmód-adatok találhatóak. Ezeket a bejegyzéseket kizárólag a bejelentkezett kliensek hozhatják létre a saját fiókjukban, de a hozzájuk rendelt edzők is jogosultak a lekérdezéseikre és elemzéseikre. Minden naplóbejegyzés idegen kulccsal (client_id) kapcsolódik a users táblához. Egy bejegyzésből eltároljuk a dátumot, az alvás mennyiségét (órában), a stressz-szintet (1-10-es skálán), a vízfogyasztást (literben), az edzésidőt (percben), az edzésintenzitást, valamint az aznapi hangulatot és egy opcionális szöveges megjegyzést. Extra adatként eltárolható a lépésszám és a testsúly is. A rendszer biztosítja, hogy egy kliens egy adott napra csak egyetlen naplót rögzíthessen, elkerülve a duplikációkat.

7.3. Heti edzésterv (WeeklyPlans)

A weekly_plans tábla a szakértők által kiírt, klienseknek szóló edzésprogramokat perzisztálja. Új tervet csak edzői jogosultsággal rendelkező felhasználó hozhat létre, illetve módosíthat, a kliensek ezeket az adatokat csak olvasási (Read) joggal érhetik el. A tábla a flexibilitás érdekében egy hibrid tárolási megoldást alkalmaz: bár relációs tábláról van szó, maga a napokra lebontott konkrét program egy JSON formátumú stringként (plan_data) kerül mentésre. Ezzel a módszerrel a heti feladatok elrendezése dinamikus maradhat anélkül, hogy minden egyes naphoz külön oszlopot kellene létrehozni. Eltároljuk továbbá a hét kezdődátumát (amely

minden esetben hétfőre kell, hogy essen), a terv nehézségi szintjét, és természetesen a tulajdonos kliens azonosítóját (`client_id`).

7.4. Meghívó (Invites)

Az `invites` tábla a rendszer zárt, biztonságos regisztrációját felügyeli. Mivel kliensként csak edzői invitálással lehet csatlakozni, ebben a kollekcióban tárolódnak az ideiglenes belépési engedélyek. Eltároljuk a meghívót generáló edző azonosítóját, az opcionálisan megadott kliens email címet, valamint a kriptográfiailag biztonságos, 32 bájtos véletlenszerű tokent. Minden meghívónak van egy lejárat ideje (`expires_at`), amely a generálástól számított 24 óra, továbbá egy logikai mező (`used`) jelzi, ha a tokent már felhasználták egy sikeres regisztráció során, meggátolva a linkek többszörös felhasználását.

7.5. Adatsere objektumok (Pydantic sémák)

Bár az adatbázis fizikai szerkezetét az SQLAlchemy modellek határozzák meg, a rendszer belső felépítésének kritikus részét képezik a Pydantic sémák. Ezek nem fizikai táblák, hanem szigorúan típusos adatsere-interfészek (DTO – Data Transfer Object). Ezek felelnek a frontend felől érkező JSON HTTP kérések (Request Body) validálásáért. Ide tartoznak például a `DailyLogCreate` vagy a `ProfileUpdate` sémák. Szerepük, hogy a nyers adatok még az adatbázis-műveletek és az AI prompt generálás előtt típusellenőrzésen essenek át, garantálva, hogy például a "vízfogyasztás" mezőbe csakis érvényes lebegőpontos szám (Float) kerülhessen.

7.6. Adatbázis-kapcsolat és munkamenet-kezelés (Session Management)

Az alkalmazás adatbázis-motorja a pehelysúlyú, fájlalapú SQLite (sqlite:///./boosted.db), amely fejlesztési és prototipizálási fázisban kiváló teljesítményt nyújt. Mivel a FastAPI alapvetően aszinkron és többszálú (multi-threaded) kéréseket szolgál ki, a kapcsolat inicializálásánál (engine) kulcsfontosságú volt a `check_same_thread=False` paraméter megadása, amely megakadályozza, hogy a szálak közötti adatbázis-hozzáférés zárolási (lock) hibákat okozzon. A kérések (HTTP Requests) életciklusa alatt az adatbázis-munkameneteket a FastAPI beépített függőséginjektálási (Dependency Injection) rendszere kezeli. A `get_db()` generátor függvény garantálja, hogy minden beérkező API hívás egy friss, izolált adatbázis-munkamenetet (Session) kapjon, majd a kérés lefutása után – a finally blokknak köszönhetően – a kapcsolatot biztonságosan lezáruljon, elkerülve a memóriaszivárgást.

7.7. AI Kontextus-objektumok (AI DTOs)

A hagyományos adatbázis-sémák mellett a rendszer speciális átmeneti objektumokat is használ a harmadik féltől származó szolgáltatásokkal való kommunikációra. A mesterséges intelligencia (Gemini) integrációjához dedikált Pydantic sémák készültek (AIDashboardRequest, ClientAnalysisRequest, ChatRequest). Ezek a sémák nem kerülnek mentésre a relációs adatbázisban; egyetlen céljuk, hogy a frontendről érkező, nyers promptokat és a bejelentkezett felhasználó szerepkörét (`user_type`) becsomagolják, validálják, majd kontextusként adják át a Generative AI modell számára.

8. Biztonság és Jogosultságkezelés

Egy modern webalkalmazás esetében a felhasználói adatok védelme és az illetéktelen hozzáférések megakadályozása kritikus fontosságú, különösen egy olyan platformnál, amely érzékeny biometrikus és életmódbeli adatokat kezel. A Boosted alkalmazás egy többrétegű (multi-layered) biztonsági modellt alkalmaz, amely a hálózati kommunikációtól kezdve az adatbázison át egészen a kliensoldali felületig biztosítja a rendszer integritását.

8.1. Adatvédelem és Kriptográfia

A felhasználók jelszavait a rendszer soha nem tárolja egyszerű szöveges (plain text) formátumban. Az autentikációs folyamat során a jelszavak titkosításáért a Python beépített hashlib könyvtára felel. A rendszer a rendkívül biztonságos SHA-256 (Secure Hash Algorithm 256-bit) egyirányú hashelési algoritmust használja, amely garantálja, hogy egy esetleges adatbázis-szivárgás esetén is visszafejthetetlenek maradjanak a jelszavak. Bejelentkezéskor a szerver a beérkező jelszót újra hasheli, és csak az eredményeket (hash lenyomatokat) hasonlítja össze.

A zártkörű regisztrációs folyamat védelmét a Python secrets kriptográfiai modulja biztosítja. Amikor egy edző új klienst hív meg, a rendszer a secrets.token_urlsafe(32) módszerrel egy 32 bájtos, kriptográfiaiilag biztonságos, véletlenszerű URL tokent generál. Ez megakadályozza, hogy illetéktelenek kitalálható linkek (pl. növekvő azonosítók) megadásával férjenek hozzá a regisztrációs felülethez.

8.2. Szerepköralapú hozzáférés vezérlés(RBAC)

A rendszer a jogosultságokat Szerepköralapú Hozzáférés-vezérlés (Role-Based Access Control) segítségével kezeli. A felhasználókat a role mező alapján két fő csoportra (Edző / Kliens) osztjuk, a be nem jelentkezett felhasználókat pedig Vendégként kezeljük. A Backend API végpontok és a Frontend felületek szigorúan izolálják az adatokat a szerepkörök között.

Jelmagyarázat: C = Létrehozás (Create), R = Olvasás (Read), U = Módosítás (Update), D = Törlés (Delete)

Szerepkör	Fiók / Profil	Napi Napló	Heti edzésterv	Kliens meghívó	Rendszer statisztika
Vendég	C (Regisztráció)	-	-	R (Token validáció)	-
Kliens	RU (Saját)	CR (Saját)	R (Saját)	-	-
Edző	RU (Saját), R(Kliensé)	R (Klienseié)	CRU (Klienseié)	C	R (Saját csapat)

8.ábra: Objektumokhoz és erőforrásokhoz való hozzáférés.

8.3. Bemeneti adatok validációja (Pydantic)

A NoSQL adatbázisokba épített "szabályok" helyett a FastAPI a Pydantic könyvtárat használja a bejövő HTTP kérések (Request Body) ellenőrzésére. Ez a köztes réteg garantálja a sémaalapú validációt.

```
class ClientRegister(BaseModel):
    token: str
    email: str
    password: str
    first_name: str
    last_name: str
```

```
class DailyLogCreate(BaseModel):
    client_id: int
    date: str
    sleep_hours: int
    stress_level: int
    water_liters: float
    workout_minutes: int
    mood: str
    notes: str = None
    workout_intensity: int = 0
    steps: int = None
    weight: float = None
```

8.3.ábra: Pydantic adatsémák és típusvalidáció a backend kódban.

A fenti ábrán látható sémák biztosítják, hogy az adatbázisba csak típushelyes (például sleep_hours esetén kizárólag egész szám, email esetén megfelelő formátumú sztring) adat kerülhessen. Ha a kliensoldalról érkező adat hibás, a Backend automatikusan elutasítja a kérést

egy 422-es (Unprocessable Entity) HTTP hibakóddal, megakadályozva az injektálásos támadásokat és az adatbázis korrupcióját.

8.4. Hálózati biztonság

A böngészők beépített védelmének kezelésére a rendszer implementálja a FastAPI CORSMiddleware (Cross-Origin Resource Sharing) komponensét. Mivel a webalkalmazás szétválasztott architektúrára épül (a React Frontend és a FastAPI Backend külön porton vagy szerveren fut), ezen szabályok definiálása elengedhetetlen. A middleware szabályozza, hogy mely külső domainek (allow_origins), HTTP metódusok és fejlécek kommunikálhatnak az API végpontokkal, védelmet nyújtva a jogosulatlan külső szkriptekkel szemben.

8.5. Kliensoldali Védelem

A biztonságra a Frontend oldalon (React) is kiemelt figyelem hárul. A felhasználók már az űrlapok kitöltésekor azonnali, interaktív hibaüzeneteket és HTML5 validációt (pl. required, type="email") kapnak, ami javítja a felhasználói élményt és csökkenti a szerver felesleges terhelését.

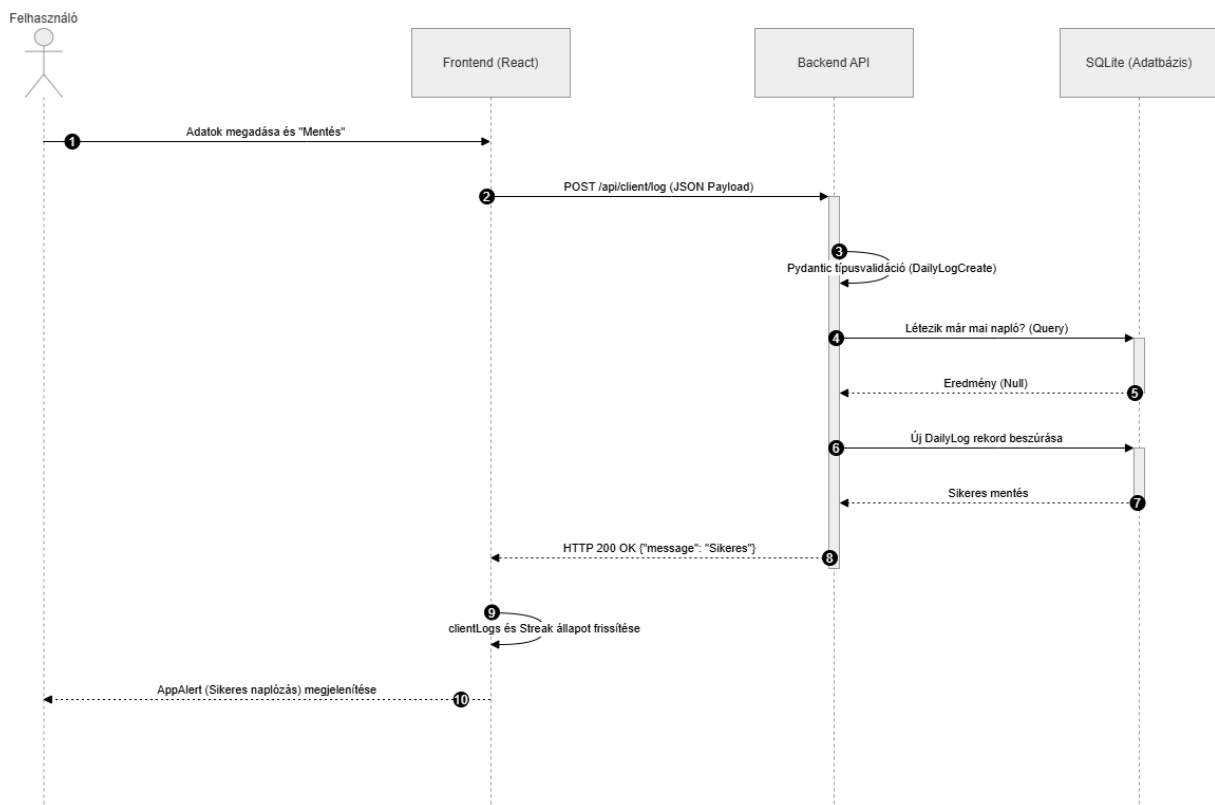
Bizonyos oldalakra (pl. DashboardView) való navigálás védve van az illetéktelen (Vendég) hozzáféréstől. A React komponens állapotváltozói (userRole és userId) működnek "Auth Guard" (Hitelesítési Védvonal) gyanánt: amennyiben a felhasználó nem rendelkezik érvényes munkamenettel (session), a rendszer automatikusan a publikus bemutatkozó vagy a bejelentkezési oldalra irányítja, elrejtve a belső üzleti logikát és a felületeket.

9. A rendszer magasszintű folyamatai

A Boosted webalkalmazás modern, aszinkron kliens-szerver architektúrára épül. Az alkalmazás magasszintű folyamatai során a felhasználói interakciók nem közvetlenül módosítják az adatbázist, hanem egy szigorúan szabályozott adatáramlási (Data Flow) láncon mennek keresztül a frontenden, a backend végpontokon, a validációs rétegeken és az ORM motoron keresztül. Ebben a fejezetben a rendszer két legkritikusabb logikai folyamatát mutatom be.

9.1. A napi biometrikus naplózás folyamata

A napi naplózás a kliensek leggyakoribb interakciója a platformmal. Ennek a folyamatnak garantálnia kell a gyorsaságot, a hibás adatok kiszűrését, valamint a napi duplikációk megakadályozását.



9.1.ábra: A napi naplózás folyamatát bemutató UML szekvenciadiagram.

A 9.1-es ábrán látható a napló rögzítésének szekvenciadiagramja. A folyamat a kliensoldali modális ablak (DailyLogModal) megnyitásával és az adatok (alvás, stressz, víz, stb.) csúszkákon történő megadásával kezdődik. Amikor a felhasználó a "Mentés" gombra kattint, a React komponens handleSaveLog aszinkron függvénye állítja össze a JSON adatcsomagot

(Payload). A rendszer automatikusan hozzárendeli a kéréshez a mai dátumot és a bejelentkezett kliens azonosítóját (`client_id`).

A kérés egy HTTP POST metódussal érkezik be a FastAPI backend `/api/client/log` végpontjára. Mielőtt az üzleti logika elindulna, a beérkező adat a `DailyLogCreate` Pydantic sémán halad keresztül, amely elvégzi a szigorú típusvalidációt. Ha az adatok helyesek, az adatbázis-munkamenet (`SessionLocal`) lekérdezi, hogy az adott felhasználó a mai napon rögzített-e már adatot. Duplikáció esetén a szerver egy 400-as HTTP hibával tér vissza.

Sikeres ellenőrzés után az SQLAlchemy ORM létrehozza az új `DailyLog` objektumot és elmenti az SQLite adatbázisba. A folyamat érdekessége, hogy ha a felhasználó megadta a testsúlyát, az algoritmus nemcsak a naplót hozza létre, hanem automatikusan frissíti a `users` táblában szereplő `current_weight` attribútumot is. Végezetül a szerver sikeres választ küld, a Frontend pedig újratöltés nélkül (State frissítéssel) frissíti a grafikont és a "Széria" számlálót, majd egy zöld `AppAlert` értesítéssel jelez a felhasználónak.

9.2. Az AI prediktív elemzés folyamata.

A rendszer legösszetettebb folyamata a Google Gemini mesterséges intelligencia integrációja, amely a nyers adatbázis-rekordokat vizuálisan értelmezhető, interaktív tanácsokká alakítja át.

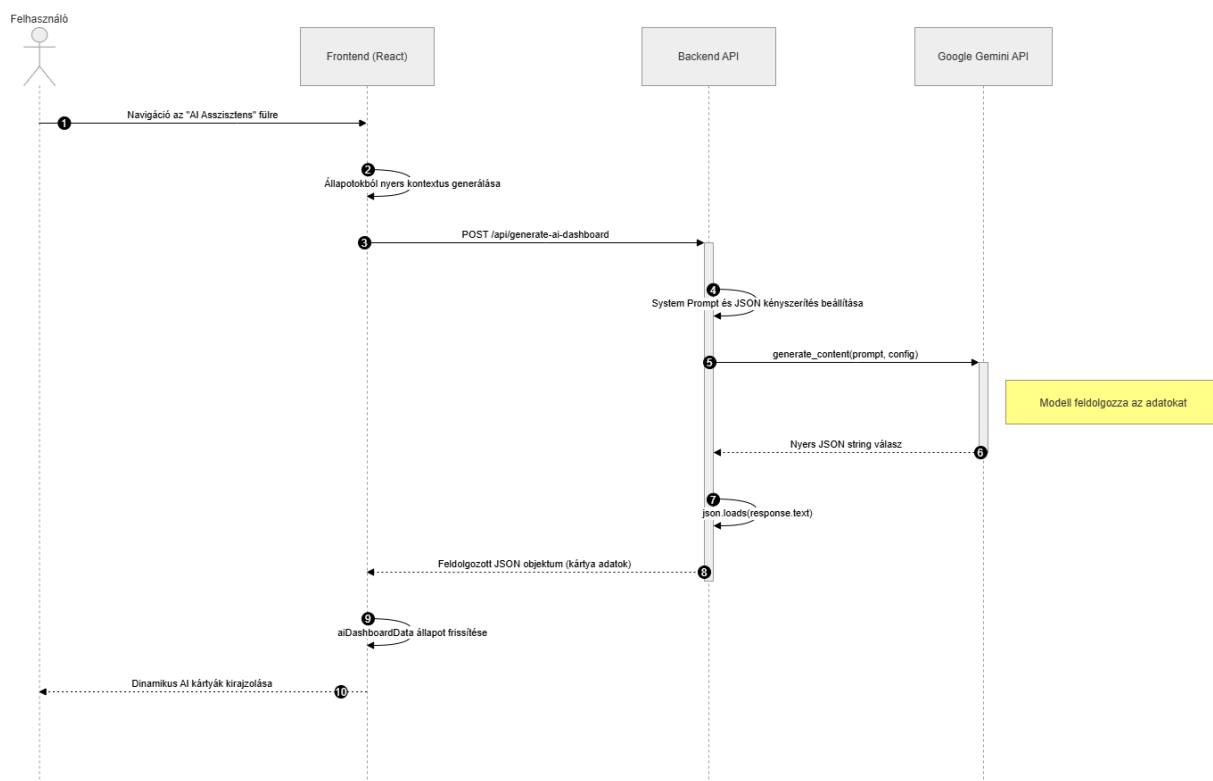
Amint a felhasználó az "AI Asszisztens" fülre navigál, a React `useEffect` hookja elindítja a `fetchAiDashboard` folyamatot, és a képernyőn megjelenik egy töltőképernyő. A Frontend az adott felhasználó szerepköre alapján felépíti az AI kontextusát (Context Data): edzők esetén a csapat létszámát, az átlagos stressz-szintet és a riasztásokat, kliens esetén pedig a célokat és az utolsó három napi napló nyers adatait fűzi össze egy nyers szöveggé.

A backend a `/api/generate-ai-dashboard` végponton fogadja a kérést. Itt a FastAPI inicializálja a `gemini-1.5-flash` modellt, majd a System Prompt (rendszerutasítás) segítségével definiálja az AI személyiségét és kényszeríti a kimeneti formátumot. A `generation_config` objektumon keresztül a backend előírja, hogy az AI válasza kizárólag érvényes JSON formátumú lehet (`response_mime_type: "application/json"`).

A Google szervereivel való aszinkron kommunikáció után a kapott nyers JSON sztringet a Python beépített `json.loads` metódusa alakítja vissza szótár (dictionary) objektummá, majd

Boosted: Edzés menedzsment webalkalmazás

továbbítja a kliensnek. A Frontend a visszaérkező adatokból (pl. `card1_title`, `summary_text`) felépíti a Bento Grid elrendezésű dinamikus kártyákat, elrejtve a töltőképernyőt.



9.2.ábra: Az adatalapú AI Dashboard generálásának szekvenciadiagramja.